

MODULE-10:

Here's a comprehensive explanation of the key points in the provided PDF:

Module Overview

- This module focuses on **implementing monitoring, elasticity, and high availability** in AWS cloud architectures.
- It covers monitoring resources, scaling compute and database resources, using load balancers and Route 53 for high availability, and applying the AWS Well-Architected Framework.
- The module uses a **café scenario** to illustrate these concepts, showing how a fictional café business utilizes various AWS services to meet its needs.

Monitoring Resources

- **Monitoring is crucial for distributed applications** to ensure timely responses, manage latency, and identify issues quickly.
- **CloudWatch** plays a central role in monitoring by:
 - Collecting metrics and logs from AWS services.
 - Supporting both built-in and custom metrics.
 - Calculating statistics and displaying them visually.
 - Providing alarms for event-driven responses.
- **CloudWatch Logs** offers centralized log management, enabling you to monitor, store, access, and analyze log files from various sources.
- **CloudWatch metrics** are data points with units, organized within namespaces. Dimensions provide additional characteristics for a metric.
- **CloudWatch alarms** trigger actions like notifications or Auto Scaling adjustments based on metric thresholds.
- **CloudWatch dashboards** offer customizable views of metrics and alarms, providing a centralized monitoring interface.
- **EventBridge** acts as an event bus, routing events between applications and services, facilitating event-driven architectures.
- **EventBridge uses event buses and pipes:** Event buses route events from multiple sources to targets, while pipes handle point-to-point integrations.
- **EventBridge rules** determine event routing based on patterns or schedules. Targets can be in different accounts or regions.
- AWS provides tools like **Cost Explorer, Budgets, and Cost and Usage Report** to track and manage infrastructure costs.

Scaling Compute Resources

- **Reactive architectures are essential** for modern applications that handle massive data, demanding high availability, and low latency.

- **Elasticity** is the infrastructure's ability to adapt to changing capacity needs. **Scaling**, the process of adjusting compute capacity, is key to achieving elasticity.
- **Vertical scaling** involves replacing resources with larger ones, while **horizontal scaling** adds or removes resources.
- **Amazon EC2 Auto Scaling** manages groups of EC2 instances, automatically adjusting capacity based on policies, schedules, and load balancer health checks.
- Auto Scaling offers benefits like **fault tolerance, availability, and cost management**.
- **An Auto Scaling group** has capacity settings (minimum, maximum, desired), launch templates defining instance configurations, and integration with load balancers.
- **Horizontal scaling with Auto Scaling groups** ensures the right number of instances are running, scaling out (adding) or in (removing) as needed.
- **Auto Scaling mechanisms** include:
 - **Scheduled actions:** Scaling based on predefined dates and times, ideal for predictable workloads.
 - **Dynamic policies:** Scaling based on monitored metrics, suited for spiky workloads.
 - **Predictive policies:** Scaling based on historical traffic patterns and machine learning, for predictable workloads.
- **AWS Auto Scaling** uses scaling plans to configure multiple resources across services like Aurora, EC2, ECS, and DynamoDB.
- **AWS Application Auto Scaling** scales resources for various AWS services, including Lambda, SageMaker, and ElastiCache, using target tracking, step, or scheduled scaling.

Scaling Databases

- **AWS database services offer various scaling mechanisms** tailored to their specific characteristics.
- **Aurora** offers:
 - **Vertical scaling** by changing instance types or storage size.
 - **Horizontal scaling** using read replicas.
 - **Automatic storage scaling managed by AWS.**
- **Aurora Serverless** provides:
 - **Horizontal compute scaling** using Aurora Compute Units (ACUs) to adjust memory and compute resources.
 - **Automatic storage scaling.**
- **Amazon RDS** allows:
 - **Vertical scaling** by resizing instances and storage volumes.
 - **Horizontal scaling** by adding read replicas.
- **DynamoDB** provides:
 - **Horizontal scaling** through on-demand mode (scaling based on actual reads/writes) or provisioned mode (setting RCUs/WCUs).
 - **Automatic storage scaling.**
- **DynamoDB auto scaling** utilizes Application Auto Scaling to dynamically adjust provisioned throughput for tables and global secondary indexes.

Using Load Balancers for High Availability

- **Highly available systems** minimize downtime and disruptions, ensuring continuous service availability.
- **Elastic Load Balancing (ELB)** distributes traffic across multiple targets (e.g., EC2 instances) to enhance availability and handle traffic fluctuations.
- ELB offers four types of load balancers:
 - **Application Load Balancer:** For HTTP and HTTPS traffic, operating at the application layer.
 - **Network Load Balancer:** For TCP, UDP, and TLS traffic, operating at the transport layer, offering ultra-low latency.
 - **Gateway Load Balancer:** For third-party virtual appliance fleets using the GENEVE protocol, operating at the network layer.
 - **Classic Load Balancer:** For legacy EC2-Classical networks, operating at both the transport and application layers.
- **Load balancer components** include:
 - **Listeners:** Check for connection requests on specific ports and protocols.
 - **Rules:** Determine how traffic is routed to targets based on conditions and actions.
 - **TLS certificates:** Ensure secure communication over HTTPS and TLS.
- **Target groups** route requests to registered targets (like EC2 instances) and utilize health checks to monitor target availability.
- **Combining Application Load Balancers with Amazon RDS Multi-AZ** eliminates single points of failure, providing high availability across multiple Availability Zones.
- **Network Load Balancers** can be used to expose centralized services, handle millions of requests per second, and support various connection methods.
- **Gateway Load Balancers** provide security solutions by enabling traffic inspection through virtual appliances, ensuring security and compliance.

Using Route 53 for High Availability

- **The Domain Name System (DNS)** translates human-readable domain names into numerical IP addresses used by computers.
- **DNS lookups involve resolvers** (recursive DNS services) that either have cached DNS information or query authoritative DNS servers.
- **Route 53** is a highly available and scalable DNS service offered by AWS.
- **Route 53 functions include:**
 - Domain registration: Purchase and management of domain names.
 - DNS routing: Directing traffic to resources based on various policies.
 - Health checking: Monitoring the health of resources like web servers.
- **Route 53 supports various routing policies:**
 - **Simple routing:** Standard DNS records without special routing features.
 - **Weighted routing:** Distributing traffic across multiple resources based on assigned weights.
 - **Latency routing:** Directing traffic to resources in the region with the lowest latency.

- **Failover routing:** Routing traffic to a healthy resource, switching to a backup if the primary one fails.
- **Geoproximity routing:** Routing traffic based on the geographical location of users and resources.
- **Geolocation routing:** Serving content based on the user's geographical location.
- **Multivalue answer routing:** Returning multiple values (like IP addresses) for DNS queries, prioritizing healthy resources.
- **IP-based routing:** Fine-tuning routing based on specific IP address ranges.
- **Route 53 hosted zones can be public (for internet traffic) or private (for traffic within a VPC).**
- **Records within hosted zones define routing rules** using various record types (e.g., A, AAAA, CNAME, MX, NS).
- **Route 53 Resolver** automatically handles DNS queries for local VPC domain names, private hosted zones, and public domains.
- **Multi-region failover** with Route 53 ensures continuous availability by routing traffic to a healthy region if one becomes unavailable.
- **Active-passive failover** uses a primary resource and a secondary one on standby, switching to the secondary if the primary fails.

AWS Well-Architected Framework and High Availability

- The **AWS Well-Architected Framework** provides best practices and guidance for building secure, resilient, efficient, and cost-effective cloud solutions.
- **Best practices for highly available systems** focus on failure management and compute/hardware considerations.
- **Failure management best practices** include:
 - **Deploying workloads to multiple locations** (Availability Zones or Regions) for redundancy and fault tolerance.
 - **Automating recovery** for components constrained to a single location, ensuring resilience.
 - **Failing over to healthy resources** automatically or manually to maintain service availability.
 - **Sending notifications when events impact availability** to enable timely responses and proactive issue resolution.
- **Compute and hardware best practices** emphasize:
 - **Dynamic scaling of compute resources** using various approaches (target tracking, predictive scaling, schedule-based, service scaling) to adapt to changing demand.

Module Wrap-up

- The module summary reiterates the key learning objectives related to implementing monitoring, elasticity, and high availability in AWS.
- It prompts reflection on how the café lab addressed the business needs of the fictional café.

- The module concludes with a knowledge check and a sample exam question to assess understanding and application of the concepts.

This breakdown explains all the important points in the provided PDF. If you have any specific questions about any part, feel free to ask!